



CI/CD

Guía Práctica para Ingeniería de
Software

Departamento de TIC · Infraestructura lasLab

Universidad Icesi · 2026

01

Módulo 1: Scripts de Automatización

02

Módulo 2: Contenedores con Docker

03

Módulo 3: Flujos de Git (Git-Flow)

04

Módulo 4: CI/CD con GitHub Actions

Sección 01

Scripts de Automatización

Buenas prácticas, ejecución segura y parametrización en Bash

- **set -e:** Detiene la ejecución si falla cualquier comando.
- **set -u:** Trata el uso de variables no definidas como error fatal.
- **set -o pipefail:** Detecta errores intermedios en tuberías (|).
- **Idempotencia:** El script debe poder ejecutarse múltiples veces sin inconsistencias.

Principio de Robustez

Un script de producción debe abortar inmediatamente ante estados inesperados para prevenir corrupciones en el servidor.

Validación de Parámetros

- Validar argumentos posicionales (\$1, \$2).
- Proveer valores por defecto con sintaxis `${VAR:-default}`.
- Verificar existencia de rutas con `[-d "$DIR"]`.

Control y Notificaciones

- Uso de códigos de salida claros (`exit 1`).
- Colores en consola para trazabilidad de éxito y fallo.
- Registro de eventos en archivos de log dedicados.

set -euo pipefail

Directiva de Ejecución Defensiva

“La automatización confiable elimina el error humano en los despliegues repetitivos de software.”

02

Contenerización con Docker

Multi-Stage Builds, Docker Compose y Reverse Proxy

- **Etapla Builder:** Descarga dependencias y compila el binario con JDK o Node.js.
- **Etapla Runtime:** Copia únicamente el artefacto final a una imagen ligera (JRE o Nginx Alpine).
- **Seguridad:** Ejecutar contenedores con usuarios no privilegiados (USER app).
- **Eficiencia:** Reduce imágenes de 800MB a menos de 40MB.

Aislamiento de Capas

Separar compilación y ejecución previene vulnerabilidades y acelera la transferencia de imágenes por la red.

Definición de Servicios

- Declaración declarativa de servicios, redes y volúmenes.
- Aislamiento en redes puente (*bridge networks*).
- Parámetros de reinicio automático (*unless-stopped*).

Mapeo de Puertos y Variables

- Exposición local segura (127.0.0.1:9088:80).
- Centralización de secretos mediante archivos `.env`.
- Control de dependencias mediante `depends_on`.

- **Punto Único de Entrada:** Nginx escucha en el puerto 80 del servidor.
- **Enrutamiento Modular:** Inclusión dinámica de `/home/iaslab/*/nginx.conf`.
- **Proxy Pass:** Reenvía el tráfico hacia el contenedor interno (`http://127.0.0.1:puerto`).
- **Preservación de Rutas:** Redirección 301 con `$is_args$args`.

Patrón Institucional Ias-Lab

Cada proyecto gestiona su propio archivo de configuración de Nginx sin afectar a otros servicios del host.

Sección 03

Estrategias de Git y GitFlow

Topología de ramas, SemVer y control de calidad

Ramas de Larga Duración

- **main / master:** Código estable en producción (v1.0.0).
- **develop:** Integración continua de nuevas características (SNAPSHOT).
- **release / qa:** Estabilización y pruebas previas al despliegue (RC).

Ramas de Soporte Temporal

- **feature/*:** Desarrollo de requerimientos específicos.
- **hotfix/*:** Corrección urgente de bugs críticos en producción.
- Integración mediante Pull Requests con revisión obligatoria.

- **SemVer (X.Y.Z):** *Major* (rompimiento), *Minor* (funcionalidad), *Patch* (corrección).
- **feat:** Introduce una nueva característica funcional.
- **fix:** Soluciona un bug o defecto identificado.
- **ci / docs / refactor:** Cambios en pipelines, guías o estructura interna.

Trazabilidad Total

Un historial estandarizado permite generar changelogs automatizados y facilita el rastreo de versiones.

Pull Requests

Flujo de Integración Protegido

“La rama principal debe permanecer inmutable; todo cambio debe pasar por revisión por pares y pruebas automatizadas.”

04

CI/CD con GitHub Actions

Pipelines condicionales y Runners Auto-Hospedados

- **Ubicación:** Archivos YAML en `.github/workflows/`.
- **Eventos (on):** Disparadores por `push`, `pull_request` o `workflow_dispatch`.
- **Jobs:** Unidades de trabajo que se ejecutan en runners específicos.
- **Steps:** Pasos secuenciales que invocan acciones predefinidas o comandos de shell.

Automatización Declarativa

Permite codificar el ciclo de vida completo de compilación, pruebas y entrega en el mismo repositorio.

Rama main (Despliegue Estático)

- Filtro en trigger:
`on.push.branches: [ main ]`.
- Copia archivos estáticos a
`/home/iaslab/app`.
- Valida y recarga Nginx del host.

Rama docker (Contenedores)

- Filtro en trigger:
`on.push.branches: [ docker ]`.
- Ejecuta `docker compose up -d --build`.
- Configura reverse proxy hacia el puerto del contenedor.

- **Seguridad Saliente:** Conexión por sondeo HTTPS (*long-polling*) hacia GitHub sin abrir puertos entrantes.
- **Acceso Directo:** Ejecución nativa en el servidor con permisos de Docker y bases de datos locales.
- **Servicio Systemd:** Servicio persistente en segundo plano bajo el usuario iaslab.
- **Etiquetas:** Seleccionable mediante `runs-on:`
`[ self-hosted ]`.

Ventajas Institucionales

Permite ejecutar pipelines directamente sobre la infraestructura de la universidad sin límites de minutos en la nube.

grid100

Self-Hosted Runner en Operación

“El runner en grid100 conecta GitHub directamente con la infraestructura interna de lasLab de forma segura y automatizada.”

1. Scripts con set -euo pipefail
2. Dockerfiles Multi-Stage
3. Ramas de GitFlow y SemVer
- 4. Workflows Condicionales en Actions**